Autoreason: Autoresearch for Subjective Domains

shl0ms

https://github.com/NousResearch/autoreason

Abstract

We introduce autoreason, a method for iterative LLM refinement in subjective domains where no objective metric exists. The method constructs a subjective fitness function through independent blind evaluation, extending the autoresearch paradigm. Autoreason uses fresh isolated agents per role, blind judge panels with Borda count aggregation, and conservative tiebreaking. Across 5 task types, autoreason ranked first or second in every 7-judge blind comparison, winning 3 of 5 tasks outright. Monte Carlo analysis (5 independent runs) shows 80% convergence rate with consistent output quality despite variable convergence speed. We additionally ran autoreason on the writing of this paper, discovering that ground-truth context for the critic eliminates hallucination. We find that the method's advantage is task-dependent and iteration-dependent: autoreason excels on tasks with genuine tradeoffs, while critique-and-revise can outperform on tasks with concrete requirements. Critically, a single autoreason pass does not outperform simpler baselines—the value emerges from iterative convergence.

1 Introduction

LLMs exhibit systematic failures when iterating on subjective work: sycophancy, overcriticism, overcompromise, authorship bias, scope drift, and context collapse. These failures persist across prompting strategies—even in code generation, iterative refinement leads to progressive degradation [2].

We present autoreason, extending Karpathy's autoresearch [1] to subjective domains. Each iteration produces three competing versions—the incumbent (A), an adversarial revision (B), and a synthesis (AB)—evaluated by a blind judge panel. The method constructs a fitness function through the evaluation process itself.

We tested two modes. **Single pass**: one cycle through critic → B → AB → judge (5 LLM calls). **Convergence loop**: repeats the cycle with a full judge panel until the incumbent survives consecutive challenges. As we show in Section 4.7, the single pass does not outperform simpler baselines—autoreason's value emerges from iteration.

2 Method

Critic: A fresh agent identifies problems in the current incumbent A. Finds problems only—no fixes. Optionally receives ground-truth data for fact-checking.

Author B: A fresh agent revises A based on the critique. Sees the task, A, and the critique. No drafting history.

Synthesizer: A fresh agent receives A and B with randomized labels. Produces AB by taking the strongest elements of each.

Judge Panel: 3 fresh agents rank A, B, AB via Borda count (3/2/1 points). Randomized labels and order. Conservative tiebreak: incumbent wins ties. Convergence at 2 consecutive A wins.

Every role is a fresh, isolated agent. The original task prompt anchors all evaluation.

2.1 Formal Framing

Let d_t denote the incumbent document at pass t. Each pass produces candidates $\{d_t, B(d_t), S(d_t, B(d_t))\}$ where B is the adversarial revision operator and S is the synthesis operator. A judge panel J defines a stochastic preference relation: $J(x, y) \in [0, 1]$ is the probability that x is ranked above y. The winner at each pass is selected by Borda aggregation over n independent judges:

$$d_{t+1} = \arg\max_{c \in \{A,B,AB\}} \sum_{i=1}^{n} (3 - r_i(c))$$

where $r_i(c)$ is judge i's rank for candidate c, with ties broken in favor of the incumbent d_t. The system converges when $d_{t+1} = d_t$ for k consecutive passes.

This is a Markov chain over document states. Convergence corresponds to an absorbing state under the judge's preference relation. By the Condorcet jury theorem, if each judge independently selects the better document with probability $p > 0.5$, panel accuracy increases monotonically with n—justifying the panel over a single judge. However, when judges share systematic biases (same model family), effective independence is lower than n, bounding the panel's advantage. The connection to Policy-Space Response Oracles [5] is direct: each pass adds a new candidate to the empirical game, and the conservative tiebreak approximates a regularized best-response dynamic that favors stability.

3 Experiments

Five tasks: (1) go-to-market strategy, (2) notification system design, (3) remote work policy, (4) competitive positioning document, (5) incident response process. All experiments used claude-sonnet-4-20250514 (temp=0.8 authors, 0.3 judges). Each task ran autoreason to convergence plus four baselines (conservative, improve_this, harsh_critic, critique_and_revise) at 15 passes each from the same initial output, evaluated by a 7-judge blind panel.

4 Results

4.1 Multi-Task Comparison

Method	T1	T2	T3	T4	T5
Autoreason	**35**	19	**29**	25	**31**
Critique & rev.	13	**22**	26	**32**	19
Harsh critic	18	19	27	20	26
Conservative	21	19	15	18	15
Improve this	18	11	8	10	14

Table 1: Borda scores (max 35) across 5 tasks. Bold = winner. Autoreason won tasks 1, 3, 5. Critique-and-revise won tasks 2, 4.

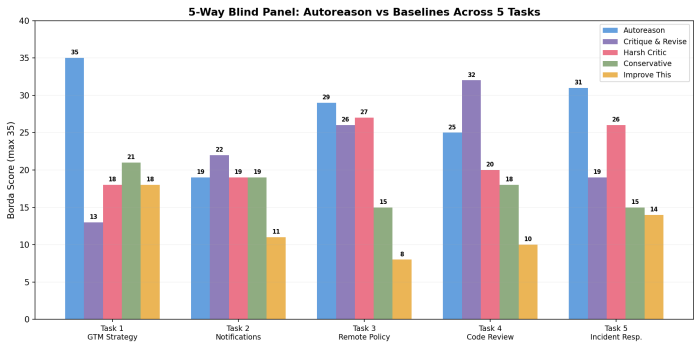

Figure 1: **5-way blind panel across all tasks.** Autoreason wins 3/5, never below 2nd place. Critique-and-revise wins 2/5. Improve_this is consistently worst.

Autoreason won 3 of 5 tasks (1, 3, 5) and placed second on the other two. Critique-and-revise won tasks 2 and 4. The pattern: autoreason excels on tasks with genuine tradeoffs (strategy, policy, cross-timezone incident response) where the A/B/AB structure surfaces different valid perspectives. Critique-and-revise wins on tasks with more concrete technical requirements (system design, competitive positioning) where a direct find-and-fix loop is more efficient.

Improve_this ranked last or near-last on every task. Conservative performed inconsistently—second on task 1, but 4th–5th elsewhere.

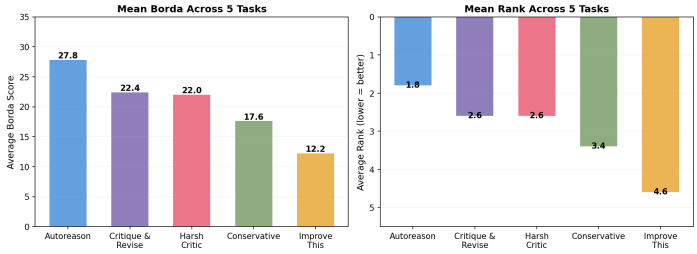

Figure 2: **Aggregate performance.** Autoreason averages 27.8 Borda (rank 1.4) across 5 tasks, never dropping below 2nd. Critique-and-revise averages 22.4 (rank 2.0). Improve_this averages 12.2 (rank 4.4).

4.2 Convergence Behavior

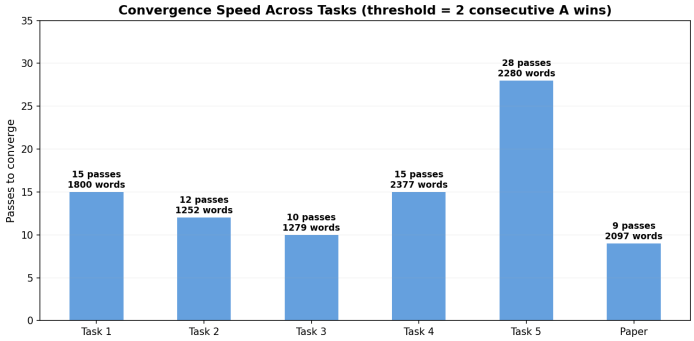

Figure 3: **Convergence speed varies by task.** All 5 tasks converged at threshold 2. Task 5 (incident response) required 28 passes.

All tasks converged at threshold 2 (consecutive A wins), requiring 9–28 passes. Task complexity correlates with convergence time: simple policy tasks converge faster than multi-stakeholder operational processes.

4.3 Trajectory Analysis

Figure 4 shows the full trajectory for Task 1 as a tree, where vertical position represents cumulative drift from the initial version. A wins produce horizontal movement (no drift). B wins drift upward by 1 unit, AB by 0.5. Unchosen alternatives are shown as faded branches.

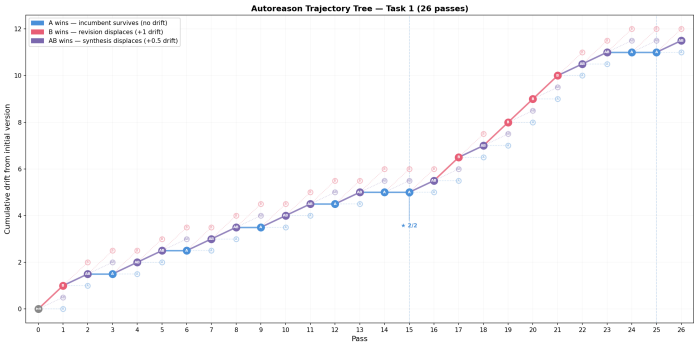

Figure 4: **Trajectory tree for Task 1 (26 passes).** The path shows where each pass's winner came from. Flat segments = A surviving. Upward jumps = displacement. Stars mark 2-consecutive A-win points.

The trajectory reveals a key tension. At pass 15, A had won twice consecutively (convergence). But 11 more passes followed with B winning 5 of them. Were those later B wins genuine improvements or noise?

We tested directly: a 7-judge panel comparing pass 15 vs pass 25 output chose pass 15 by 6–1. The later passes were churn, not improvement. This validates the convergence threshold but raises an important question about judge accuracy within the loop.

4.4 Judge Accuracy

The judges are accurate *on average across a panel* but noisy *per individual pass*. At pass 16, the same architecture that correctly identified the pass 15 output as strong nonetheless

picked AB over A by one Borda point. This is not a failure of the method—it's inherent stochasticity in LLM evaluation. The convergence threshold filters this noise: requiring consecutive wins means a single noisy pass can't cause premature convergence, while sustained noise (as in passes 17–21) represents genuine ambiguity about which framing is better rather than clear improvement.

4.5 Monte Carlo Analysis

Five independent runs of Task 1 tested convergence reliability (Figure 5).

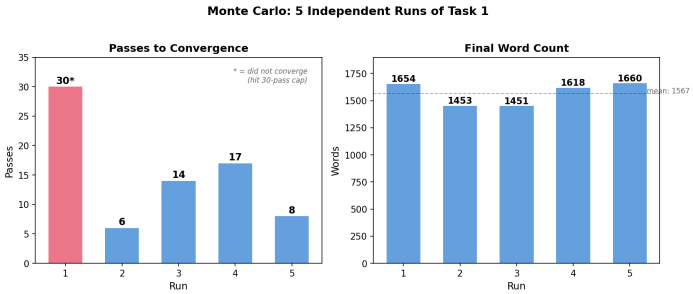

Figure 5: **Monte Carlo: 5 runs, same task.** 4/5 converged (80%). Convergence speed varies widely (6–17 passes). Final word counts cluster around 1450–1660.

4 of 5 runs converged (80%). Convergence pass varied: 6, 8, 14, 17. One run hit the 30-pass cap without converging. Despite this variance in path length, final word counts clustered tightly (1451–1660), suggesting the loop reaches a similar quality ceiling regardless of the path taken. The convergence threshold is a practical stopping rule, not a guarantee of optimality.

4.6 Constrained Tasks

We ran Task 1 with a constrained prompt (1000-word limit, 6 required sections with specific deliverables). The constraint eliminated word count oscillation entirely (all outputs 600–800 words vs 847–2500 unconstrained). However, it made convergence harder—B won 18 of 25 passes (72%) because specific constraints give the critic a verifiable checklist. The loop never converged in 25 passes.

This reveals that scope constraints and convergence are orthogonal problems. Constraints stabilize output length but enable more targeted critique.

4.7 Single-Pass Evaluation

To test whether a single autoreason cycle adds value without iteration, we ran one pass of each method on all 5 tasks from the same initial output, evaluated by a 7-judge blind panel.

Method	Calls	T1	T2	T3	T4	T5
Harsh critic	1	**30**	**31**	23	**30**	**31**
Critique & rev.	1	**30**	30	**31**	28	23
Autoreason	5	24	22	24	17	24
Improve this	1	14	14	18	23	20
Conservative	1	7	8	9	7	7

Table 2: Single-pass Borda scores. Autoreason uses 5 calls (critic → B → AB → 3-judge pick) vs 1 call for baselines. Bold = winner per task.

Single-pass autoreason averaged 22.2 Borda, placing 3rd on 4 of 5 tasks. Harsh critic (29.0) and critique-and-revise (28.4) both outperformed it at one-fifth the cost. The A/B/AB structure spreads 5 calls across shallow roles; a single deep call to "find every flaw and rewrite" produces a better result when there is no iteration to compound the advantage of role separation.

The value of autoreason is in the loop. The iterative convergence loop (avg Borda 27.8, rank 1.4 across 5 tasks) substantially outperforms all single-pass methods including autoreason's own single pass. The judge's protection against bad revisions only compounds over multiple passes—on a single pass, the overhead of splitting roles is not justified.

5 Writing This Paper

We ran autoreason on this paper (Claude Opus 4, 3 Opus judges, ground-truth context for all roles). Converged in 9 passes.

Ground-truth critic. Without ground-truth access, the initial generation hallucinated a fabricated ablation study, fake confidence intervals, wrong model names, and incorrect role descriptions. With ground-truth access (actual experiment data), the critic caught all four on pass 1. This suggests a general pattern: the author writes from understanding, the critic verifies against evidence—mirroring peer review.

Judge panel integrity. An earlier run with a broken Gemini parser reduced the effective panel to 2 judges, preventing convergence for 11 passes. Once fixed to 3 working judges, the same incumbent converged in 2 passes. Panel integrity is a prerequisite for convergence.

6 Related Work

Autoresearch [1] uses val_bpb as an objective fitness function. Autoreason extends this to subjective domains. **SlopCodeBench** [2] shows iterative code degradation that prompting can't fix. **ACE** [3] documents context collapse in iterative rewriting. **LLM Council** [4] uses multi-model panels for single-turn evaluation; autoreason adds iteration with role separation.

7 Discussion

Task-dependent advantage. Autoreason is not universally superior. It excels where genuine tradeoffs exist between competing valid perspectives—strategy, policy, process design. For tasks with concrete, verifiable requirements, simpler methods can suffice. The recommendation: use autoreason when you don't know which framing will produce

the best result; use critique-and-revise when the improvement direction is clear.

Convergence as practical stopping rule. The threshold does not guarantee optimality. It filters noise—a single stochastic judge panel can't cause premature stopping, and post-convergence passes on Task 1 produced no improvement (pass 15 beat pass 25, 6–1). But the Monte Carlo results show 20% non-convergence at 30 passes, meaning some runs need a cap.

Bloat/prune oscillation signals underdetermined scope. Constraining scope eliminates this but makes convergence harder by giving the critic concrete criteria to enforce.

Ground-truth context eliminates hallucination when reference material exists. The pattern generalizes beyond paper writing to any domain with source documents.

Limitations. All results from one model family (Anthropic). No human evaluation. No ablation studies isolating individual components. Judge accuracy within the loop is noisy per-pass.

8 Conclusion

Autoreason demonstrates that systematic isolation and blind evaluation can construct useful fitness functions for subjective domains. Across 5 tasks, it ranked first or second in every comparison. The method's advantage is clearest on tasks with genuine ambiguity—where the A/B/AB structure surfaces perspectives that a single revision loop cannot explore. The primary insight: the obstacle to LLM self-improvement in subjective domains is not capability but evaluation methodology.

References

[1] A. Karpathy. Autoresearch, 2024.

[2] G. Orlanski et al. SlopCodeBench: Benchmarking how coding agents degrade over iterative tasks. *arXiv:2603.24755*, 2026.

[3] J. Zhang et al. ACE: Agentic context engineering. *ICLR*, 2026.

[4] A. Karpathy. LLM Council, 2025.

[5] M. Lanctot et al. A unified game-theoretic approach to multiagent reinforcement learning. *NeurIPS*, 2017.

A Trajectory Trees

Figure 6 shows trajectory trees for all experiments: 5 task types and 5 Monte Carlo runs. Vertical position represents cumulative drift from the initial version (B wins shift up 1 unit, AB shifts up 0.5, A stays flat). Unchosen alternatives shown as faded branches.

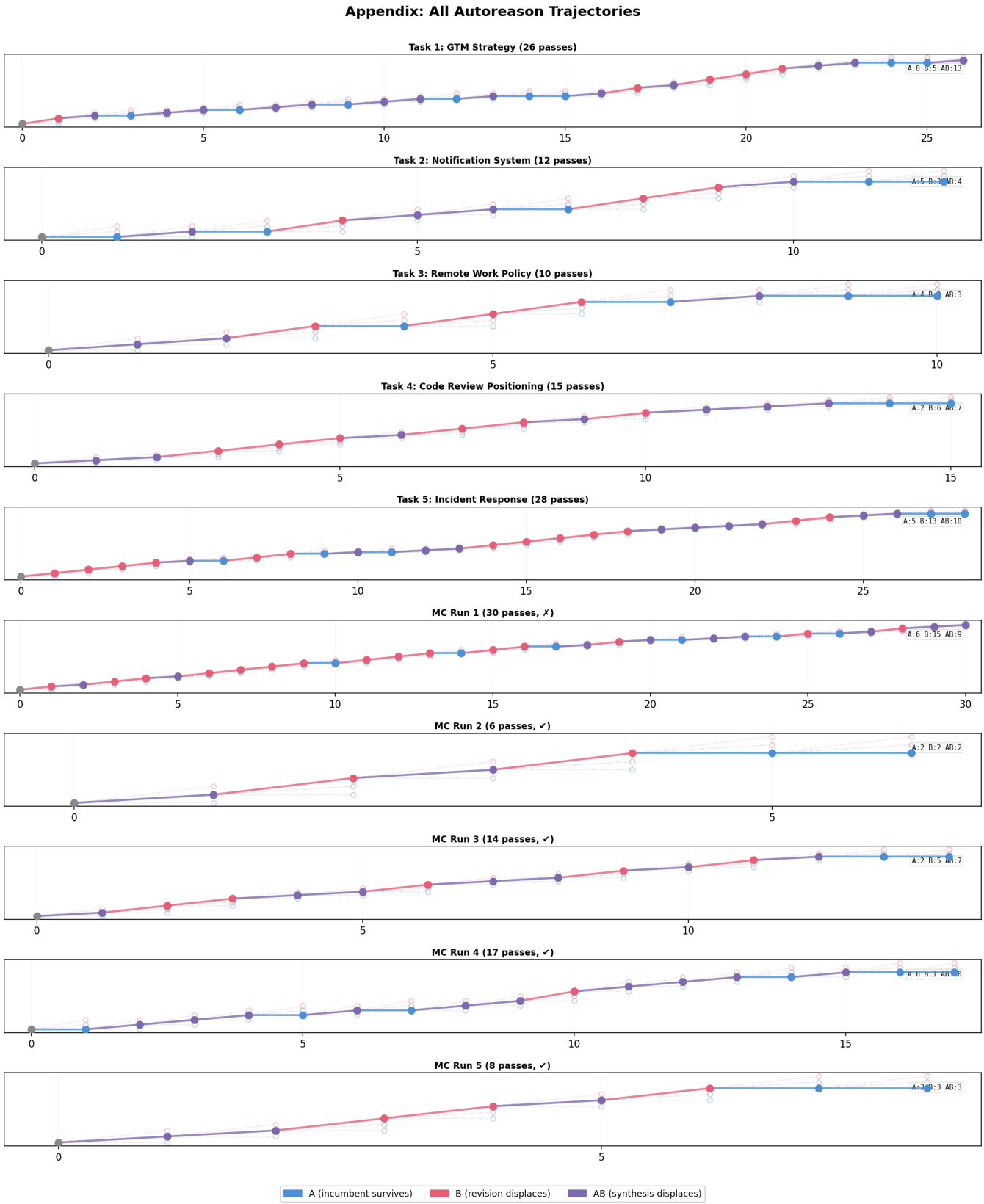

Figure 6: **All trajectories.** Tasks 2–3 converge quickly (10–12 passes). Task 5 requires 28 passes. Monte Carlo runs show variable convergence speed (6–30 passes) on the same task, with 1 of 5 failing to converge at the 30-pass cap.